



THE GUIDE

RuWork Developer Guide

A complete code explanation and walkthrough for the RuWork platform

Written for the RuWork development team — university students who know basic JavaScript but may be new to Express, MongoDB, JWTs, middleware, React, or Axios interceptors. It explains not just *what* the code does, but *why* it exists and what would break without it.

CONTENTS



1	Start here	1
2	The five documents	2
00	— Project Overview	2
01	— Backend Complete Guide	2
02	— Frontend Complete Guide	2
03	— Code Glossary	2
04	— Request and Data Flows	2
3	The ten things most worth understanding	3
4	Verified project status	4
5	Two things the code disagrees with the plan about	5
6	Repository map	6
7	Conventions used in this guide	7

Start here

IF YOU ARE...	READ
New to the project	<i>00 — Project Overview</i>
Working on the backend	<i>01 — Backend Complete Guide</i>
Working on the frontend	<i>02 — Frontend Complete Guide</i>
Stuck on a keyword or symbol	<i>03 — Code Glossary</i>
Tracing one feature end to end	<i>04 — Request and Data Flows</i>
Preparing for a viva	00 → 04 → 01 §§5–6, 19 → 03

The five documents

00 00 — *Project Overview*

What RuWork is, the three roles, every capability, what the system deliberately does *not* do (no payments, no admin message access, no destructive deletion), the technology stack explained in plain language, the full architecture diagram, the email flow, and the current verified status.

Also records two honest discrepancies between `PROJECT_PLAN.md` and the code — including a real latent bug in the seed script.

01 01 — *Backend Complete Guide*

The most detailed document. 28 sections covering folder structure, `index.js` startup order, Express fundamentals, MongoDB/Mongoose, all ten models, authentication, middleware, email verification, the password lifecycle, every domain system (jobs, applications, reviews, messaging, notifications, settings, audits), security middleware, environment configuration, error handling, logging, health, seeding, testing, six full request walkthroughs, a file-by-file table, and a common-questions section.

02 02 — *Frontend Complete Guide*

18 sections: Vite/React startup, React concepts with real RuWork examples, routing and role protection, authentication state, Axios and its interceptors, the service layer, pages vs components, shared components, all Student/Provider/Admin flows, messaging and notification UI, Tailwind and responsive design, accessibility, lazy loading, testing, and a file-by-file table.

03 03 — *Code Glossary*

A cheat sheet: JavaScript syntax (`?.`, `??`, spread, destructuring, `async/await`), Node/Express terms, MongoDB/Mongoose concepts, authentication and security vocabulary, React concepts, HTTP methods and status codes, and RuWork-specific terminology — each with a real example from the codebase.

04 04 — *Request and Data Flows*

18 end-to-end flows as Mermaid diagrams plus plain-text fallbacks, each naming the exact page → service → endpoint → router → middleware → controller → utility → model chain. Ends with a complete verified endpoint reference.

The ten things most worth understanding

1. **The server decides, not the browser.** Frontend guards are usability. Every rule is enforced again server-side. → *Backend §5.3*
2. **The JWT is verified, then the account is re-read from MongoDB.** A token is a photograph from login time; suspension and revocation happen afterwards. → *Backend §6.4*
3. **Passwords and tokens are only ever stored as hashes.** Passwords use salted bcrypt; verification and reset tokens use SHA-256 with an expiry enforced inside the query. → *Backend §7*
4. **tokenVersion makes logout real.** Clearing `sessionStorage` removes a copy; incrementing the counter kills the token. → *Backend §5.4*
5. **Uniqueness is guaranteed by database indexes, not controller checks.** Controller checks are UX; the unique index wins the race. → *Backend §4.5*
6. **Identities are always derived, never supplied.** Message recipients come from the Application; review subjects come from the Application; job ownership comes from the authenticated provider. → *Backend §§13–15*
7. **Moderation is reversible and history-preserving.** Nothing is hard-deleted, so mistakes are recoverable and audits stay meaningful. → *Backend §11.5*
8. **Identical responses prevent enumeration.** Login and forgot-password never reveal whether an account exists. → *Backend §8.2*
9. **Errors are safe by default.** Stack traces are logged with a correlation id, never returned, in any environment. → *Backend §21*
10. **There is no payment processing.** RuWork records agreed prices and nothing more. → *Overview §4*

Verified project status

Measured on master while writing these documents:

CHECK	RESULT
Backend tests	118 / 118 passing
Frontend tests	94 / 94 passing across 21 test files
Backend files passing <code>node --check</code>	57 / 57
Frontend ESLint	Passes, no errors or warnings
Production build	Succeeds; main chunk $\approx$ 377 kB ($\approx$ 119 kB gzipped)
<code>npm audit</code> (both projects)	0 vulnerabilities

Not verified, and not claimed: live MongoDB, live SMTP delivery, and real deployment. No external credentials or hosting environment are configured in this repository.

Two things the code disagrees with the plan about

Documented in full in *Overview §11*. **The code is the source of truth.**

1. **Application creation is `POST /api/jobs/:jobId/applications`** (on the Job router), not under `/api/applications` as `PROJECT_PLAN.md` implies. The plan's wording is imprecise; the design itself is sound.
2. **`scripts/seedDemo.js` has a real latent bug.** It creates Applications with `studentNote`, but `models/application.js` requires `applicationNote`. Because no MongoDB is configured, the script has never run and validation has never rejected it — `npm run seed:demo` would fail today. The fix is a two-word rename in two places. It was **not** applied here because this task is documentation-only.

Repository map

```
YENU RUHUNA/
├── PROJECT_PLAN.md                ← phase-by-phase history and specification
├── RuWork_backend-master/         ← Express + MongoDB API
│   ├── index.js  models/  controllers/  routes/
│   └── middlewares/  utils/  scripts/  tests/
├── frontend/                     ← React + Vite browser application
│   └── src/ (pages, components, services, context, hooks, utils, test)
├── docs/
│   ├── RuWork_Backend_Requirements.pdf      (original brief – unchanged)
│   ├── RuWork_Backend_Objects_Relationships.pdf (original brief – unchanged)
│   └── developer-guide/                    ← this guide
└── design/
```


Conventions used in this guide

- **File paths** are given relative to `RuWork_backend-master/` or `frontend/`, matching the guide you are reading.
- **Functions** are written as `fileName.js → functionName()`.
- **Line numbers are never cited** — they change as code evolves. Search by function name instead.
- **Code snippets are short** and used only where they genuinely clarify the logic.
- Callouts marked **Security reason:** explain a deliberate security decision.
-  marks a gotcha or a known defect.

Markdown is the authoritative format for this guide.

